VECTOR

vItaHlp

User Manual

Version 4.0.0

Authors	Philipp Kallenberg, Stefan Rombach, Peter Pokorny, Thomas Urban
Status	Released

Document Information

History

Author	Date	Version	Remarks
Philipp Kallenberg Stefan Rombach Peter Pokorny	2019-03-28	1.0.0	Initial User Documentation
Thomas Urban	2019-05-20	2.0.0	Introduced OS Test Module
Peter Pokorny	2019-07-01	2.1.0	Update description for Setup Test
Thomas Urban	2019-07-26	2.2.0	Introduced Eth Test Module
Peter Pokorny	2019-09-13 2019-10-14	3.0.0	Introduced Fls Test Module Introduced MCU Test Module
Peter Pokorny	2019-12-10 2020-01-17 2020-02-14	3.1.0	Use of vMem module in Fls Test Module Remove description of Fls Exclusive Area definitions Introduced LIN Test Module Introduced WDG Test Module
Peter Pokorny	2020-05-14 2020-05-25	4.0.0	Support LIN Slave in LIN Test Module Support test communication via Ethernet

Reference Documents

No.	Source	Title	Version
[1]	Vector	TechnicalReference_Os.pdf	See delivery information
[2]	Vector	TechnicalReference_Bre.pdf	See delivery information
[3]	Vector	UserManual_vBaseEnv.pdf	See delivery information
[4]	Vector	TechnicalReference_vLinkGen.pdf	See delivery information
[5]	Vector	TechnicalReference_Asr_MemoryMapping.pdf	See delivery information
[6]	Vector	UserManual_DaVinciTeamAndPlatformSupport.pdf	See delivery information

© 2020 Vector Informatik GmbH



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Introduction 7

2 Functional Description 8
2.1 Test Scope 8
2.1.1 Setup Test 8
2.1.2 Standard Test 9
2.2 Test Communication 9
2.2.1 Test Communication Protocol 9
2.2.2 Test Communication Link Layer 9
2.2.2.1 Test Communication over CAN 10
2.2.2.2 Test Communication over Eth 10
2.3 Watchdog Standard Procedure 10
2.4 Error Handling 10

3 Configuration 12
3.1 vItaHlp 12
3.1.1 Test Modules 12
3.1.2 Test Communication 12
3.1.3 Configuration of CANoe Network Channel Name 13
3.2 Additional Components 13
3.2.1 vBaseEnv 14
3.2.1.1 vBRS 14
3.2.1.2 EcuC 14
3.2.2 BRE 15
3.3 Additional AUTOSAR Standard Definitions 15

4 Integration 16
4.1 Scope of Delivery 16
4.1.1 Static Files of the vItaHlp ECU Application 16
4.1.2 Dynamic Files of the vItaHlp ECU Application 17
4.1.3 Dynamic Files of the vItaHlp CANoe Application 17
4.2 Critical Sections 19
4.3 Stub Modules 19
4.4 Memory Map Files 19
4.5 Compiler Configuration Files 20
4.6 CANoe Configuration 20

5 Test Modules 21
5.1 CAN 21

5.1.1 Setup Test 21
5.2 OS 22
5.3 Eth 22
5.3.1 Setup Test 22
5.4 Fls 23
5.4.1 vMem 23
5.5 MCU 24
5.6 LIN 24
5.6.1 Setup Test 24
5.7 WDG 25

6 Glossary and Abbreviations 26
6.1 Glossary 26
6.2 Abbreviations 26

7 Contact 28

Illustrations

Figure 1-1 vItaHlp in a component view 7
Figure 2-1 Example of test environment for the Setup Test 8
Figure 2-2 Example of test environment for a Standard Test 9
Figure 3-1 Example of Test Modules 12
Figure 3-2 Example of mapping between CAN controller and CANoe channel name. 13
Figure 3-3 Example of CANoe Network node overview 13
Figure 3-4 vBRS Testsuite Support container 14

Tables

Table 3-1 vItaHlp Init functions 14
Table 4-1 Static files of the vItaHlp ECU application 17
Table 4-2 Generated files of the vItaHlp ECU application 17
Table 4-3 Generated vItaHlp CANoe application files 18
Table 5-1 CAN Setup Test alternating pattern 21
Table 5-2 Eth Setup Test alternating pattern 23
Table 5-3 LIN Setup Test alternating pattern 24

1 Introduction

The Vector Integration Test Application for the Hardware Layer Packages (vItaHlp) provides

- A test application for the ECU
- An optional CANoe remote station
  - to control and observe the test application in the ECU via a dedicated test link
  - to control and observe external interfaces of the ECU like CAN bus or DIO
  - with test scripts for automated tests
- Tooling for test configuration and generation

In addition to the MCAL components under test, only a few more components (OS and basic runtime components) are needed to run the vItaHlp tests. The tests use API calls of the component drivers under test and the OS.

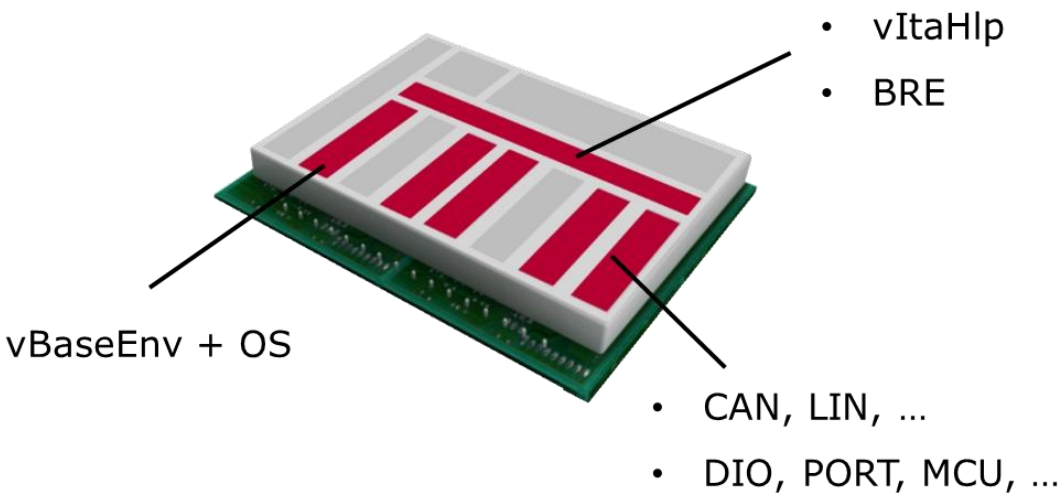


Figure 1-1 vItaHlp in a component view

2 Functional Description

2.1 Test Scope

The vItaHlp test application provides tests for hardware dependent software modules (MCAL modules). Once the startup and initialization of the ECU is finished, any of the configured vItaHlp tests may be executed.

The vItaHlp tests trigger function calls, provided by the service API of the driver modules under test, however it is not the intention of vItaHlp to cover the complete range of API calls. Each of those tests can be executed without the need of the higher layer software modules, e.g. defined by the AUTOSAR standard.

The vItaHlp provides two types of tests:

- Setup Test
- Standard Test

2.1.1 Setup Test

The Setup Test is designed, to execute a predefined action on the ECU, e.g. to transmit a predefined data pattern on the communication bus under test. The result, e.g. the data transmission, may be verified by means of message tracing tools or by oscilloscope. The action is triggered cyclically in predefined time intervals. The Setup Test starts immediately after startup of the ECU for each configured Unit Under Test, in case the initialization is finished successfully.

The test environment required for the Setup Test is depicted in Figure 2-1.

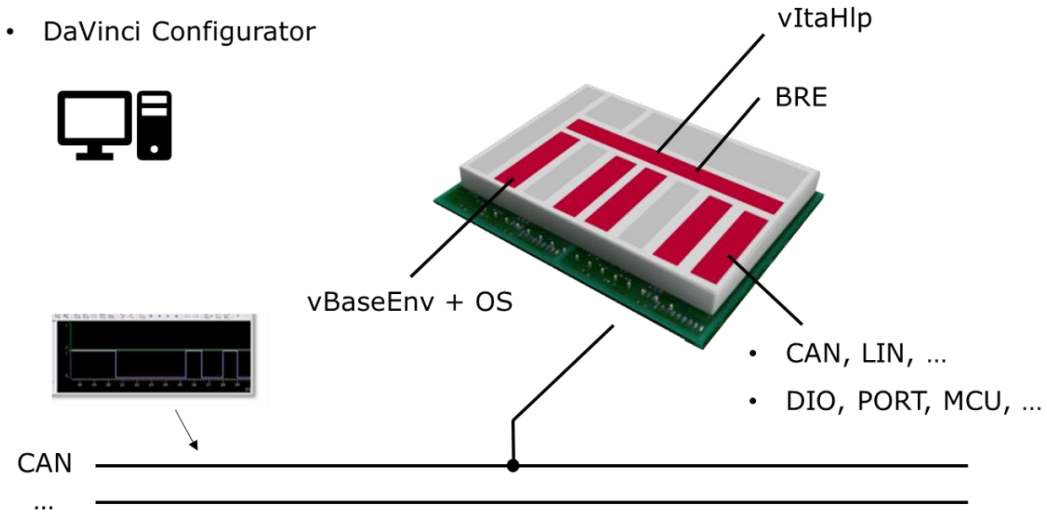


Figure 2-1 Example of test environment for the Setup Test

Note
The observed pattern on the bus may be interfered by error prevention mechanism of the communication controller (e.g. CAN error passive state, CAN bus off).

2.1.2 Standard Test

A Standard Test covers certain features of MCAL components on the target hardware. It allows a deeper testing of several service API of the component and to test more complex sequences.

The execution of the standard tests is controlled by a vItaHlp CANoe test script. The different test steps and the test outcome are logged in the corresponding CANoe test report. As a very first test control action, the Setup Test scenario is terminated at the beginning of the standard tests.

The test environment required for the standard test is depicted in Figure 2-2.

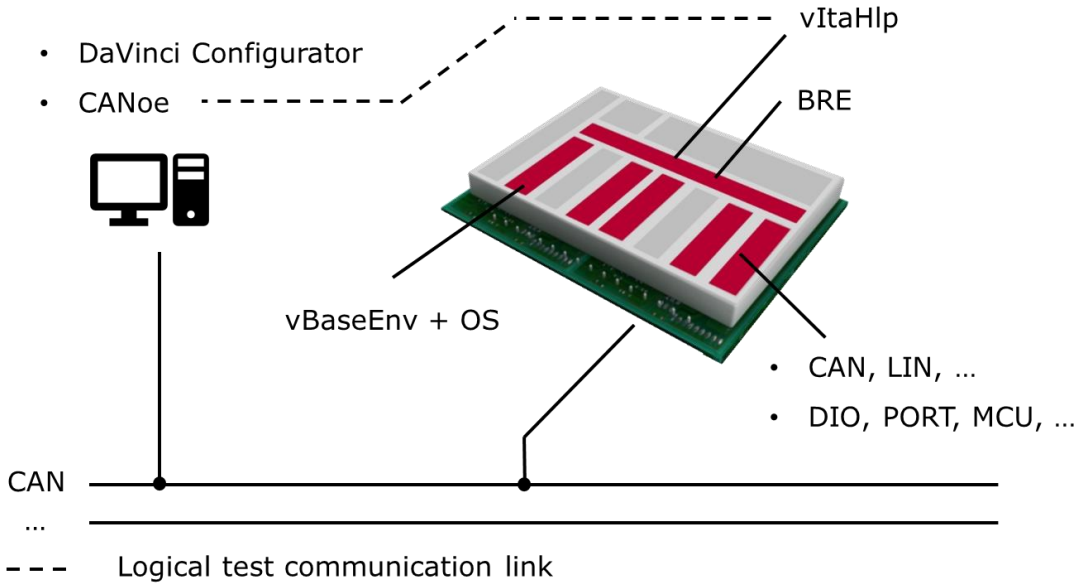


Figure 2-2 Example of test environment for a Standard Test

2.2 Test Communication

2.2.1 Test Communication Protocol

The test communication protocol is based on 8 byte messages and consists of

- Command messages
- Event messages

A command message is sent from the CANoe test control application and is used to trigger a service API call of the Unit Under Test on the ECU.

The event message is sent from the vItaHlp ECU test application back to the CANoe test control application as response to an executed test step on ECU or to indicate an error condition. Any response from the ECU is sent asynchronously to the received command messages.

2.2.2 Test Communication Link Layer

The vItaHlp test application supports different test communication link layer types.

If the test communication channel belongs to a Unit Under Test (e.g. CAN channel used for standard tests and test communication), there may be test states, in which this channel is temporarily not operational (e.g. CAN bus in sleep mode).

In order not to lose the test communication events from the DUT during this time, each event triggered by the DUT is collected first in an event queue of the vItaHlp ECU application. The event queue is flushed cyclically, as soon the test communication channel is operational.

It is the responsibility of the vItaHlp CANoe test control application, to make sure that no command is sent if the test communication channel is not established. There is no queuing mechanism for command messages.

2.2.2.1 Test Communication over CAN

One of the CAN busses of the DUT may be used for test communication purposes. For each CAN channel, the CAN Ids 0 and 1 are reserved for the test communication, regardless whether the channel is used for test communication or not.

There is no constraint in terms of the CAN Unit Under Test. A CAN controller may be defined as Unit Under Test and may be used for test communication too in parallel.

2.2.2.2 Test Communication over Eth

The test communication via an Ethernet channel of the DUT is supported by vItaHlp. Dedicated Ethernet frame type (EtherType) definitions are used for the command and event messages:

- 0xFEED for command messages
- 0xBACC for event messages

They are also different compared to the frame type used for the Eth frames of the Unit Under Test.

An Ethernet channel of the DUT may be defined as Unit Under Test and may be used for test communication too in parallel.

2.3 Watchdog Standard Procedure

If a WDG module is part of the DaVinci project configuration, this watchdog module needs to be served by one of the ECU applications. The vItaHlp test application provides a mechanism of cyclic refresh of the watchdog trigger conditions for all configured ECU watchdog modules, independent whether a watchdog module is subject of detailed tests or not. The vItaHlpWdgTestModule is not required to perform the watchdog standard procedure.

The timing conditions used for the cyclic refresh are initialized with default values at startup of the ECU.

During execution of the vItaHlp WDG standard tests, those timing values are modified for the WDG driver under test. After test, the initial values are restored again.

2.4 Error Handling

The following error types are reported to the CANoe test control application:

- Errors of the vItaHlp ECU test application

- Errors returned by a module API call
- Errors reported by a module via the Default Error Tracer (DET)

The previously described event reporting mechanism of the test communication is used for the error reporting (see section 2.2).

The occurred errors are analyzed and reported within the CANoe test result file. A test case fails, if an error event occurs which is not expected at the given time of the test execution.

3 Configuration

The vItaHlp ECU application is located above the MCAL layer of a BSW stack.

In addition to the MCAL modules some basic components are needed for the control and startup of the ECU.

Like for other MICROSAR BSW modules, the configuration of the vItaHlp ECU application is supported by the DaVinci Configurator.

3.1 vItaHlp

3.1.1 Test Modules

The vItaHlp component can be used to test certain basic functionalities of MCAL modules. For each module, which can be tested, a Test Module container exists. For modules, which may be instantiated multiple times, it is also possible to add multiple Test Module containers to the test configuration.



Figure 3-1 Example of Test Modules

Each Test Module may comprise several Test Configuration containers, as depicted in the example of Figure 3-1 for the vItaHlpCanTestConfig_Drv1_Channel1 and vItaHlpCanTestConfig_Drv1_Channel2 sub-containers.

The Test Configuration container is used to set the detailed parameters needed for a test unit:

- References to the Unit Under Test and the selection of test elements (if applicable).
- Selection of the different test aspects.

The contents of the Test Configuration containers differ between the test modules. The detailed description for each element is given in the DaVinci Configurator Properties view.

3.1.2 Test Communication

The vItaHlp test application supports different link layers for the test communication, as already described in section 2.2 of this document.

Note
The vItaHlpTestCom container shall be added to the DaVinci Configurator project, if one of the standard tests is enabled. Select the appropriate vItaHlpTestComLinkLayer type.

3.1.3 Configuration of CANoe Network Channel Name

For some test modules as well as for the test communication link layer, it is necessary to define the mapping between the communication channel on ECU and the representation within the CANoe network configuration. The recommended procedure is given using the example of the CAN test communication link layer configuration.

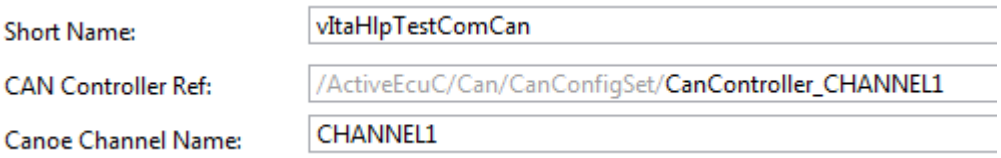


Figure 3-2 Example of mapping between CAN controller and CANoe channel name

- Select the appropriate CAN or Ethernet controller of the DUT, which should be used for the test communication messages.
- Set an appropriate CANoe channel name.

Caution
Do not use CANoe keywords (CAN1…CAN32) as channel name. In general, avoid using similar constructs like CAN0, CAN00, etc.

- Once the CANoe configuration is done (see section 4.6), the default CANoe network names shall be aligned by the user to the CANoe channel names set before in the vItaHlp ECU configuration. See the example shown in Figure 3-2 and Figure 3-3.

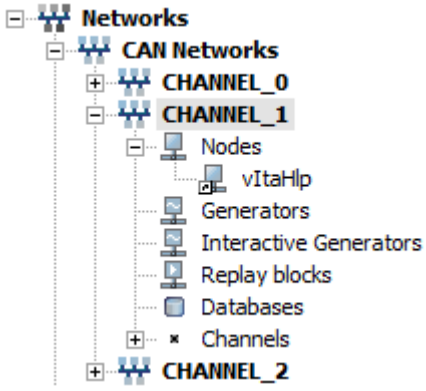


Figure 3-3 Example of CANoe Network node overview

3.2 Additional Components

The vItaHlp ECU application requires a few mandatory components. The detailed description for configuration of those components is given in the documents [1] – [4].

3.2.1 vBaseEnv

As described in [3], the vBaseEnv package always contains the following modules:

- OS (Vector MICROSAR Operating System)
- vBaseEnv (Vector Base Environment)
- vBRS (Vector Basic Runtime System)
- vLinkGen (Vector Linkerscript Generator)
- EcuC (Autosar EcuC module)

3.2.1.1 vBRS

vBRSTestsuiteSupport

The vItaHlp component requires the vBRSTestsuiteSupport container of the vBRS module. As vItaHlp uses a proprietary protocol for the test communication, the test communication support (TCC Support) of the vBRS module shall be set to value ‘None’.

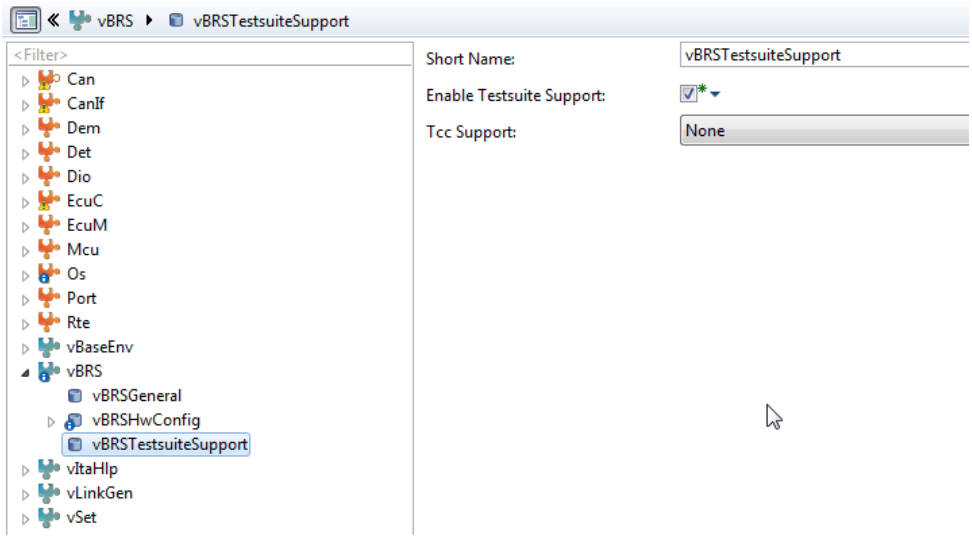


Figure 3-4 vBRS Testsuite Support container

Generation of EcuM stub module

For startup of the vItaHlp ECU application, the execution of the ECU initialization functions is required. The initialization functions related to the different initialization phases are provided by the vBRS as generated function stubs of the EcuM stub module. Enable the stub generation of the EcuM in the vBRS configuration.

3.2.1.2 EcuC

For the vItaHlp ECU application two Init Functions exist, which are part of the Init Function list of the EcuC container.

Init function name	Init phase	Associated header file
vItaHlp_PreInit	INIT_ONE_LATE	vItaHlp.h
vItaHlp_Init	INIT_TWO_IF	vItaHlp.h

Table 3-1 vItaHlp Init functions

3.2.2 BRE

The BRE functionality is configured in the DaVinci Configurator within the RTE module container. It supports a basic set of RTE functionality. It provides the Exclusive Area Handling and the execution of cyclic and background triggered schedulable entities (BSW Main-Functions) for the BSW modules, as well as for vItaHlp itself.

3.3 Additional AUTOSAR Standard Definitions

For projects using vItaHlp, the following additional AUTOSAR Standard Definitions shall be added to the DaVinci Configurator project:

- EcuM

4 Integration

4.1 Scope of Delivery

The vItaHlp package consists of files, which contain static implemented code and files, which are generated by the DaVinci Configurator.

The files for the CANoe test control application are required only in case one of the vItaHlp standard tests is configured.

4.1.1 Static Files of the vItaHlp ECU Application

File Name	Description
vItaHlp.c	General functions of vItaHlp.
vItaHlp.h	
vItaHlp_CommonUtils.c	Common utility functions used by vItaHlp.
vItaHlp_CommonUtils.h	
vItaHlp_Can.c	vItaHlp CAN test module functions.
vItaHlp_Can.h	
vItaHlp_Eth.c	vItaHlp Eth test module functions.
vItaHlp_Eth.h	
vItaHlp_Lin.c	vItaHlp LIN test module functions.
vItaHlp_Lin.h	
vItaHlp_Fls.c	vItaHlp Fls test module functions.
vItaHlp_Fls.h	
vItaHlp_Mcu.c	vItaHlp MCU test module functions.
vItaHlp_Mcu.h	
vItaHlp_Wdg.c	vItaHlp WDG test module functions.
vItaHlp_Wdg.h	
vItaHlp_ComIf.c	Dispatching of communication messages to TestCom or Unit Under Test.
vItaHlp_ComIf.h	
vItaHlp_EventQueue.c	Administration of the vItaHlp event queue for test communication.
vItaHlp_EventQueue.h	
vItaHlp_TestCom.c	Definition and implementation of link layer parts of the vItaHlp test communication.
vItaHlp_TestCom.h	
vItaHlp_WdgService.c	Definition and implementation of the watchdog standard procedure for all WDG modules configured in the project.
vItaHlp_WdgService.h	
vItaHlp_Os.c	vItaHlp OS test module functions.
vItaHlp_Os.h	
BrsTestsuite.h	The header file required by vBRS for support of test applications.
_vItaHlp_MemMap.h	Template for the Memory Section definition file ‘vItaHlp_MemMap.h’.
_vItaHlp_Compiler.h	Template for the Compiler Abstraction definition file ‘vItaHlp_Compiler.h’.

Table 4-1 Static files of the vItaHlp ECU application

4.1.2 Dynamic Files of the vItaHlp ECU Application

The dynamic files are generated by the DaVinci Configurator.

File Name	Description
vItaHlp_Cfg.c	The general configuration files of vItaHlp.
vItaHlp_Cfg.h	
vItaHlp_Can_Cfg.c	The configuration files of the vItaHlp CAN test module.
vItaHlp_Can_Cfg.h	
vItaHlp_Eth_Cfg.c	The configuration files of the vItaHlp Eth test module.
vItaHlp_Eth_Cfg.h	
vItaHlp_Lin_Cfg.c	The configuration files of the vItaHlp Lin test module.
vItaHlp_Lin_Cfg.h	
vItaHlp_Fls_Cfg.c	The configuration files of the vItaHlp Fls test module.
vItaHlp_Fls_Cfg.h	
vItaHlp_Mcu_Cfg.c	The configuration files of the vItaHlp MCU test module.
vItaHlp_Mcu_Cfg.h	
vItaHlp_Wdg_Cfg.c	The configuration files used by the vItaHlp WDG test module and vItaHlp WDG standard procedure.
vItaHlp_Wdg_Cfg.h	
vItaHlp_TestCom_Cfg.c	The configuration files of the vItaHlp test communication link layer.
vItaHlp_TestCom_Cfg.h	
vItaHlp_Os_Cfg.c	The configuration files of the vItaHlp OS test module.
vItaHlp_Os_Cfg.h	

Table 4-2 Generated files of the vItaHlp ECU application

4.1.3 Dynamic Files of the vItaHlp CANoe Application

All files required for the vItaHlp CANoe test control application will be generated when at least one of the standard tests is configured. The files are stored in the application specific CANoe directory.

File Name	Description
vItaHlp.can	The CAPL code source file.
vItaHlp_UT_General.cin	The included CAPL code source file for general test module functions.
vItaHlp_UT_EventDispatcher.cin	The included CAPL code source file for event dispatching routine.
vItaHlp.vsysvar	The CANoe System Variables definition file.
vItaHlpTestScript.vxt	The CANoe XML test script.
vItaHlp_UT_TestCommunication.cin	The included CAPL code source file for test communication routines.

vItaHlp_BusIf_Cfg.cin	The included CAPL code source file for routines to dispatch the incoming traffic on tester to test communication message handler or LT message handler for the Unit Under Test.
vItaHlp_UT_CAN.cin	The included CAPL code source file for test communication of CAN test module.
vItaHlp_UT_Eth.cin	The included CAPL code source file for test communication of Eth test module.
vItaHlp_UT_Lin.cin	The included CAPL code source file for test communication of LIN test module.
vItaHlp_UT_Fls.cin	The included CAPL code source file for test communication of Fls test module.
vItaHlp_UT_Mcu.cin	The included CAPL code source file for test communication of MCU test module.
vItaHlp_UT_Wdg.cin	The included CAPL code source file for test communication of WDG test module.
vItaHlp_UT_Os.cin	The included CAPL code source file for test communication of OS test module.
vItaHlp_LT_Can_Rx.cin	The included CAPL code source file for CAN module Rx tests.
vItaHlp_LT_Can_Tx.cin	The included CAPL code source file for CAN module Tx tests.
vItaHlp_LT_Can_Rx_Cfg.cin	The included CAPL code configuration source file for CAN module Rx tests.
vItaHlp_LT_Can_Tx_Cfg.cin	The included CAPL code configuration source file for CAN module Tx tests.
vItaHlp_LT_Eth_Rx.cin	The included CAPL code source file for Eth module Rx tests.
vItaHlp_LT_Eth_Tx.cin	The included CAPL code source file for Eth module Tx tests.
vItaHlp_LT_Eth_Rx_Cfg.cin	The included CAPL code configuration source file for Eth module Rx tests.
vItaHlp_LT_Eth_Tx_Cfg.cin	The included CAPL code configuration source file for Eth module Tx tests.
vItaHlp_LT_Lin_Tx.cin	The included CAPL code source file for LIN module Tx tests.
vItaHlp_LT_Lin_Rx.cin	The included CAPL code source file for LIN module Rx tests including functions used for LIN Master on tester side (ECU LIN Slave).
vItaHlp_LT_Lin_Cfg.cin	The included CAPL code configuration source file for LIN module tests.
vItaHlp_UT_Fls_Cfg.cin	The included CAPL code configuration source file for Fls module tests.

Table 4-3 Generated vItaHlp CANoe application files



Note
To use the CANoe test application, the generated files shall be integrated in a CANoe project file by the user. See section 4.6.

4.2 Critical Sections

The vItaHlp has critical code sections which must not be interrupted.

To achieve consistent preparation steps in more complex test scenarios of the vItaHlp CAN Test Module, the following function calls are used:

- SchM_Enter_vItaHlp_VITAHLP_CAN_EXCLUSIVE_AREA
- SchM_Exit_vItaHlp_VITAHLP_CAN_EXCLUSIVE_AREA

Any Write Access to the vItaHlp event queue is enclosed by call of the following functions:

- SchM_Enter_vItaHlp_VITAHLP_EVENTQUEUE_EXCLUSIVE_AREA
- SchM_Exit_vItaHlp_VITAHLP_EVENTQUEUE_EXCLUSIVE_AREA

Any Write Access to the vItaHlp test communication link state is enclosed by call of the following functions:

- SchM_Enter_vItaHlp_VITAHLP_TESTCOM_EXCLUSIVE_AREA
- SchM_Exit_vItaHlp_VITAHLP_TESTCOM_EXCLUSIVE_AREA

4.3 Stub Modules

During execution of several module test scenarios, upper layer interface functions are called. As the corresponding interface modules are not part of the vItaHlp ECU application, those functions need to be provided by stub modules.

The vBRS module provides all the required stub modules. A summary of available stub modules and description of the stubbed functions is given in [3].

4.4 Memory Map Files

All memory section definitions used by vItaHlp are provided in the template file '_vItaHlp_MemMap.h', which shall be adapted by the user according to the project needs.



Edit
Include the adapted and renamed template file ‘vItaHlp_MemMap.h’ in your project specific ‘MemMap.h’ file.

```
#include "Os_MemMap.h"
#include "vItaHlp_MemMap.h"
```

4.5 Compiler Configuration Files

All compiler abstractions used by vItaHlp are provided in the template file ‘_vItaHlp_Compiler.h’, which shall be adapted by the user according to the project needs.



Edit
Include the adapted and renamed template file ‘vItaHlp_Compiler.h’ in your project specific ‘Compiler_Cfg.h’ file.

```
/**********************************************************************************
 * INCLUDES
 **********************************************************************************/
#include "vItaHlp_Compiler.h"
```

4.6 CANoe Configuration

The following instructions describe step-by-step how to integrate the generated CANoe files in a CANoe project. The files are described in section 4.1.3.



Practical Procedure

- Create a Network Node (e.g. called “vItaHlp”) within the CANoe Simulation Setup window.
- Add CAPL-Code ‘vItaHlp.can’ as Node specification file to this Network Node.
- Assign the corresponding bus types to this Network Node.
- Check that the CANoe network channel names are equal to the CANoe channel names set in the vItaHlp ECU configuration (see section 3.1.3).
- Create a new CANoe test environment (e.g. called “vItaHlp”) and assign an XML Test Module.
- Assign the ‘vItaHlpTestScript.vxt’ file as Test Script to this configuration.
- Load the ‘vItaHlp.vsysvar’ file as Referenced File in the System Variables configuration menu.

5 Test Modules

For each BSW module, which can be tested with vItaHlp, a test module is provided. The general concept for the test module configuration is described in section 3.1.1. The following sub sections contain only test module specific information, which is not part of the general description.

Within the test configuration of a test module, standard tests can be activated or deactivated. Depending on the test it can be chosen which object shall be considered for the tests. For detailed information refer to the description of the configuration parameters in the configuration tool.

5.1 CAN

For CAN a setup test and multiple standard tests are available.

Those tests can be configured for each controller of a CAN driver, as represented by the vItaHlp configuration hierarchy for CAN:

> vItaHlp Test Modules

> Test Module CAN ⇔ one per CAN driver

> Test Configuration ⇔ one per CAN controller of this driver

5.1.1 Setup Test

The setup test can be instantiated for each CAN controller of the CAN driver configuration, by choosing an arbitrary Tx hardware object of a certain controller within its Test Configuration.

For all configured controllers the test application in the ECU transmits the following 8 Byte pattern cyclically right after start-up:

Alternating:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte 6	Byte7
0x00	0x00	0x00	0x00	0xFF	0xFF	0xFF	0xFF
0xFF	0xFF	0xFF	0xFF	0x00	0x00	0x00	0x00

Table 5-1 CAN Setup Test alternating pattern

If the CAN controller is in error free state, this alternating pattern can be observed using a tracing tool like CANoe or an oscilloscope.

If due to an error situation, the transmission of the same CAN frame is repeated by the controller HW, it is evident that only a fixed pattern can be observed which should correspond to the content shown in one of the above table columns.

Besides the pattern vItaHlp assigns different CAN IDs for each tested controller during test generation. The used CAN ID for a certain controller can be found in the generated file ‘vItaHlp_Can_Cfg.c’.

Thus, it can be verified that not only the pattern is received, but also that the channel assignment between SW/HW is as expected.

5.2 OS

For OS multiple standard tests are available.

Those tests can be configured for each Core of the OS, as represented by the vItaHlp configuration hierarchy for OS:

> vItaHlp Test Modules

> Test Module OS

> Test Configuration ⇔ one per OS Core

5.3 Eth

For Eth a setup test and multiple standard tests are available.

Those tests can be configured for each controller of an Eth driver, as represented by the vItaHlp configuration hierarchy for Eth:

> vItaHlp Test Modules

> Test Module Eth ⇔ one per Eth driver

> Test Configuration ⇔ one per Eth controller of this driver



Note
For some HW platform types (e.g. RH850), the user application needs to provide the implementation of Eth driver callout functions (e.g. Appl_Eth_30_Etnb_Wait()).

Please refer to the documentation of the Eth driver module for details.



Note
Depending on the test environment (e.g. Evaluation board type), it may be necessary to execute a few preparation steps for Hw components, before the Eth driver can be fully initialized (e.g. release an Eth transceiver from Reset state).

The vItaHlp_Eth test module supports the implementation of such preparation steps within a generated ‘User block’ in vItaHlp_Eth_Cfg.c. The content of the ‘User block’ is not overwritten during generation of this file.

5.3.1 Setup Test

The setup test will be instantiated for each Eth controller of the Eth driver configuration. There is no need to configure additional elements for the setup test.

For all configured controllers the test application in the ECU transmits the following predefined 46 Byte pattern cyclically right after start-up:

Alternating:

Byte0	Byte1	Byte2	Byte…	Byte…	Byte43	Byte 44	Byte45
0x00	0x01	0x02	…	…	0x2B	0x2C	0x2D

0xFF	0xFE	0xFD	…	…	0xD4	0xD3	0xD2

Table 5-2 Eth Setup Test alternating pattern

This first pattern is starting with Byte0 and a value of 0x00 and increments the value with each following byte by one. The second pattern is starting with Byte0 and a value of 0xFF and decrements the value with each following byte by one.

If the Eth controller is in error free state, this pattern can be observed using a tracing tool like CANoe or an ethernet network analyzer.

Each Eth controller transmits the test pattern with its own ‘Physical Address (MAC-Address)’ as source address and the broadcast address ‘FF:FF:FF:FF:FF:FF’ as destination address. As frame type the ‘0xBEEF’ is used.

Thus, it can be verified that not only the pattern is received, but also that the channel assignment between SW/HW is as expected.

5.4 Fls

For Fls multiple standard tests are available.

Those tests can be configured for each sector of the Fls module, as represented by the vItaHlp configuration hierarchy for Fls:

> vItaHlp Test Modules

> Test Module Fls (only a single instance is supported)

> Test Configuration ⇔ one per Fls sector



Note
In case the Link process fails due to less RAM memory, it is recommended to review the configuration of the vItaHlp Fls Standard Test Elements.

In this case, it may be necessary to reduce either the number of Fls Standard Test Elements, or the number of Fls pages defined for a test element.

It is mandatory to enable the Fls_GetJobResult API of the Flash driver module under test.

The configuration of FlsJobEndNotification and FlsJobErrorNotification is optional. If no notification function is configured for the Fls module, vItaHlp evaluates the outcome of a flash driver job by cyclic call of the Fls_GetJobResult function. A combination of both mechanisms is supported as well. The cyclic call of the Fls_GetJobResult function is part of the vItaHlp_MainFunction().

5.4.1 vMem

The vMem module and corresponding vMemAccM may be tested using the vItaHlp Fls test module, if additional the HW independent MICROSAR BSW module Fls_30_vMemAccM is configured. The Fls_30_vMemAccM module behaves like an AUTOSAR Flash driver module towards the upper layer (here vItaHlp Fls test module).

5.5 MCU

For MCU a single test configuration container is supported, in which multiple standard tests can be configured:

> vItaHlp Test Modules

> Test Module MCU

> Test Configuration MCU



Note
For the MCU test ‘Perform Reset’ the detailed reset reason is not evaluated by the vItaHlp test script, as the set of reset types provided by Mcu_ResetType may vary between different HW platforms.

The value of the reset reason is printed in the test report. It is up to the tester, to verify that the reset reason is meaningful for the given test steps.

5.6 LIN

For LIN a setup test is available for LIN channels configured as LIN Master on ECU. Multiple standard tests are available for LIN channels configured as LIN Master or SLAVE on ECU.

Those tests can be configured for each channel of a LIN driver, as represented by the vItaHlp configuration hierarchy for LIN:

> vItaHlp Test Modules

> Test Module LIN ⇔ one per LIN driver

> Test Configuration ⇔ one per LIN channel of this driver

5.6.1 Setup Test

The setup test is instantiated for each LIN Master channel of the LIN driver configuration.

For all configured LIN Master channels, the test application in the ECU transmits the following 8 Byte pattern cyclically right after start-up:

Alternating:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte 6	Byte7
0x00	0x00	0x00	0x00	0xFF	0xFF	0xFF	0xFF
0xFF	0xFF	0xFF	0xFF	0x00	0x00	0x00	0x00

Table 5-3 LIN Setup Test alternating pattern

If the LIN channel is in error free state, this pattern can be observed using a tracing tool like CANoe or an oscilloscope.

For the setup test vItaHlp assigns the same LIN PID for each tested LIN channel during test generation. The used LIN PID is defined in the static file ‘vItaHlp_Lin.c’.

5.7 WDG

For WDG multiple standard tests are available.

Those tests can be configured for each WDG in the configuration, as represented by the vItaHlp configuration hierarchy for WDG:

- vItaHlp Test Modules
  - Test Module WDG
    - Test Configuration ⇔ one per WDG

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
3rd party components / MCAL	BSW components that have been provided by a company other than Vector. Vector may have integrated the software within the SIP but does not take over any responsibility with regard to the functionality of these components.
CANoe	Tool for simulation and testing of networks and electronic control units. CANoe is a product of Vector Informatik GmbH.
Critical Section	A critical section is a sequence of instructions where mutual exclusion must be ensured. Such a section is called ‘critical’ because shared data is modified within it.
DaVinci Configurator	Vectors configuration and generation tool of the MICROSAR components. Sometimes also referenced as DaVinci Generator, if the ability of code generation shall be highlighted.
Lower Tester	Operates on a controllable/observable external visible communication path like a channel (CAN, FlexRay…). Here data can be monitored, stimulated.
Upper Tester	Operates on the Service Level of the ECU software and thus needs an additional communication path (e.g. Test Communication).

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
ASR	Automotive Open System Architecture (AUTOSAR)
BRE	Basic Run-Time Environment (the Vector solution which supports a basic set of RTE functionality)
BSW	Basic Software
CAN	Controller Area Network
CAPL	Communication Access Programming Language
DET	Default Error Tracer, AUTOSAR BSW component
DIO	Digital Input/Output
DUT	Device Under Test (the ECU under test)
ECU	Electronic Control Unit
EcuC	ECU Configuration; configuration of all BSW as well as MCAL components (Note: AUTOSAR also supports EcuC splits containing just a subset of components with its configuration)
EcuM	ECU State Manager; AUTOSAR BSW component
Eth	Ethernet
Fls	Flash Driver module, AUTOSAR BSW component

HLP	Hardware Layer Package; MICROSAR BSW components, as part of the SIP with strong dependencies to the used hardware platform.
LIN	Local Interconnect Network
LT	Lower Tester
MCAL	Microcontroller Abstraction Layer
MCU	Microcontroller Unit, AUTOSAR BSW component
OS	Operating System, AUTOSAR component
PID	Protected ID
PORT	PORT Driver module, AUTOSAR BSW component
RTE	Run-Time Environment, AUTOSAR component
SIP	Software Integration Package (as provided by Vector); MICROSAR delivery containing customers' selection of MICROSAR BSW components including all parts of MCAL Integration.
UT	Upper Tester
vBRS	Vector Basic Runtime System
vItaHlp	Vector Integration Test Application for Hardware Layer Package.
vMem	Vector Memory (Driver)
vMemAccM	Vector Memory Access Manager
WDG	Watchdog
XML	Extensible Markup Language

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2020 Vector Informatik GmbH